

simple, if a name for it was provided in the page code. But, as the *Submit* button's name is missing, it can be found by selecting the items of type button and choosing the first among the resultant collection. This is clicked by calling the click method!

```
button = browser.find_by_tag('button').first
button.click()
```

Next, in the resultant page, the keyword research is searched for and if it is found, then the message 'Yes' is provided; otherwise, the message you get is 'No'.

When you execute the following code in a proxy environment...

```
if browser.is_text_present('Research'):
    print("Yes, the Article Found")
else:
    print("No, it wasn't found")
```

... then the corresponding settings need to be made in the code, as shown below:

```
proxyIP = 'specify your proxy IP here'
proxyPort = specify your proxy port here
proxy_settings = {'network.proxy.type': 1,
                  'network.proxy.http': proxyIP,
                  'network.proxy.http_port': proxyPort,
                  'network.proxy.ssl': proxyIP,
                  'network.proxy.ssl_port': proxyPort,
                  'network.proxy.socks': proxyIP,

                  'network.proxy.socks_port': proxyPort,
                  'network.proxy.ftp': proxyIP,
                  'network.proxy.ftp_port': proxyPort
                }
```

Finding elements

Splinter provides various ways of finding the elements in a Web page as listed in Table 1.

Method	Code
CSS	<code>browser.find_by_css('p')</code>
Xpath	<code>browser.find_by_xpath('//h3')</code>
Tag	<code>browser.find_by_tag('button')</code>
Name	<code>browser.find_by_name('search')</code>
Text	<code>browser.find_by_text('OSFY')</code>
Id	<code>browser.find_by_id('email')</code>
Value	<code>browser.find_by_value('zotero')</code>

Table 1: Finding elements in a page

Finding hyperlinks

To find hyperlinks, Splinter provides the functions listed in

Table 2 that can be chosen based on the nature of the data such as a 'link text', '', which is used as the input to find a link.

Method	Code
Text	<code>my_links = browser.find_link_by_text('Click to Subscribe')</code>
Partial Text	<code>my_links = browser.find_link_by_partial_text('Click to')</code>
HRef	<code>my_links = browser.find_link_by_href('http://pondiuni.edu.in')</code>
Partial HRef	<code>my_links = browser.find_link_by_partial_href('pondiuni')</code>

Table 2: Finding links

Executing JavaScript

Splinter allows the execution of JavaScript with supported browser objects. A simple code to execute JavaScript is shown below:

```
browser.execute_script("$('body').empty()")
```

Similarly, evaluation of the script can also be carried out using the following code segment:

```
browser.evaluate_script("10+70") == 80
```

Headless mode

When you wish to perform the operations without explicitly opening the browsers, then drivers such as *PhantomJS* and *Zope.testbrowser* can be used. To use these drivers, the corresponding dependencies need to be satisfied.

Handling cookies

Splinter allows you to handle cookies using simple functions. To do this, the *cookies* attribute from the browser instance is utilised.

For creating a cookie, use the following code:

```
browser.cookies.add({'country': 'India'})
```

To fetch all the cookies, use the code below:

```
browser.cookies.all()
```

For deletion of one or more cookies, use the following code:

```
browser.cookies.delete('test') # deletes the cookie 'test'
browser.cookies.delete('test1', 'test2') # deletes two
cookies test1, test2
browser.cookies.delete() # deletes all cookies
```

Handling iframes and alerts

To handle *iframes*, Splinter provides a method for the browser